

BÁO CÁO XÂY DỰNG HỆ THỐNG TOAST NOTIFICATION

Dự án: Cải thiện trải nghiệm người dùng (UX) - Phản hồi hành động

Người thực hiện: Đỗ Việt Anh

1. Hiểu bài (Overview)

- Toast là gì?** Là một dạng thông báo nhỏ (pop-up) xuất hiện tạm thời trên màn hình.
- Mục đích:** Cung cấp phản hồi ngay lập tức cho người dùng sau khi họ thực hiện một hành động (như Like, Save, Comment) mà không làm gián đoạn trải nghiệm (không chặn màn hình như alert).
- Giải quyết vấn đề:** Giúp người dùng biết hệ thống đang xử lý, đã thành công hay thất bại, đặc biệt hữu ích khi đường truyền mạng chậm.

2. Dữ liệu & Trạng thái (Data & State)

Hệ thống cần quản lý các thông tin sau:

- Hàng đợi (Queue):** Sử dụng một Mảng (Array) để lưu trữ danh sách các thông báo đang chờ được hiển thị.
- Cấu hình Toast:** Mỗi thông báo là một Đối tượng (Object) gồm: message (nội dung), type (success/error/info/warning), và duration (thời gian tồn tại).
- Trạng thái bận (isProcessing):** Biến Logic (Boolean) để đánh dấu xem có Toast nào đang hiển thị hay không, nhằm điều phối việc xếp hàng.

3. Luồng hoạt động (Workflow)

- Kích hoạt:** Người dùng bấm nút, hàm tạo Toast được gọi.
- Nạp hàng đợi:** Thông báo mới được đẩy (push) vào mảng Queue.
- Kiểm tra & Hiển thị:** Hệ thống kiểm tra nếu không có Toast nào đang hiện, nó sẽ lấy thông báo đầu tiên ra khỏi hàng.
- Tạo DOM:** JavaScript tự động tạo thẻ <div>, gán Class theo loại (Success/Error...) và chèn vào Container.
- Kết thúc:** Sau 4 giây, Toast tự xóa khỏi màn hình. Hệ thống tiếp tục kiểm tra Queue để hiện cái tiếp theo (đệ quy).

4. Pseudocode (Mã giả)

A. Luồng thêm Toast

Plaintext

HÀM requestToast(nội_dung, loại):

Tạo object mới { msg: nội_dung, type: loại }

Thêm vào mảng hàng_đợi

GỌI HÀM renderQueue()

B. Luồng xử lý Queue (Logic cốt lõi)

Plaintext

HÀM renderQueue():

NẾU (đang bận xử lý) HOẶC (hàng đợi rỗng) THÌ: Thoát hàm

Đánh dấu: Đang bận = True

Lấy Toast đầu tiên ra khỏi mảng

Tạo element HTML và hiển thị lên màn hình

ĐỢI 4000ms:

Xóa element HTML

Đánh dấu: Đang bận = False

GỌI LẠI HÀM renderQueue() (để xử lý các toast còn lại)

5. Các trường hợp biên (Edge Cases)

1. **Bấm liên tục (Spamming):** Người dùng bấm Like 20 lần/giây. Hệ thống phải đảm bảo 20 thông báo này hiện lần lượt, không "đề" ra 20 cái đề lên nhau làm nát giao diện.
2. **Nội dung quá dài:** Thông báo có thể bị vỡ khung. Cần CSS word-break hoặc giới hạn ký tự để đảm bảo thẩm mỹ.
3. **Xung đột Dark Mode:** Toast phải có độ tương phản tốt (thường dùng nền đậm, chữ trắng) để nổi bật trên cả nền sáng và tối của website.
4. **Tắt trình duyệt:** Vì hàng đợi lưu trong RAM (JS), nếu F5 trang, các thông báo chưa kịp hiện sẽ biến mất. (Chấp nhận được vì đây là thông báo tức thời).

6. Mở rộng (Scalability)

- **Nút Close (x):** Thêm sự kiện click vào dấu x để xóa Toast ngay lập tức và gọi nhanh Toast tiếp theo trong hàng đợi.
- **Đa dạng vị trí:** Có thể thay đổi vị trí top-left, bottom-center bằng cách thay đổi Class của Container.
- **Tùy chỉnh thời gian:** Lỗi (Error) nên hiện lâu hơn (6s) để người dùng kịp đọc, còn Success chỉ cần 2-3s.